

ENERGIA PARA SOBREVIVER

OS AJUSTES
LINEARES QUE
REFINAM AS
PREVISÕES DE
AUTONOMIA

5

LISTA DE FIGURAS

Figura 1 – Resultado do código-fonte 1	15
Figura 2 – Resultado do código-fonte 2	17
Figura 3 – Resultado do código-fonte 3	18
Figura 4 – Resultado do código-fonte 4	23
Figura 5 – Resultado do código-fonte 5	30

EMENDAS

LISTA DE TABELAS

Tabela 1 – Dados do desempenho de um servidor	8
Tabela 2– Exemplo de matriz X utilizada no modelo.....	11

EXEMPLO

LISTA DE CÓDIGOS-FONTE

Código-fonte 1 – Montando X e Y	14
Código-fonte 2 – Encontrando os coeficientes	16
Código-fonte 3 – Encontrando os coeficientes com <i>scikit-learn</i>	18
Código-fonte 4 – Teste t e F.....	21
Código-fonte 5 – Código com as métricas	30

EMANIP

SUMÁRIO

AJUSTE DE MODELOS LINEARES	6
1 REGRESSÃO LINEAR MÚLTIPLA	6
1.1 Expansão para múltiplas variáveis explicativas.....	7
1.2 Representação matricial.....	9
1.2.1 Vetor Y	10
1.2.2 Matriz X	10
1.2.3 Vetor β	11
1.2.4 Vetor ε	12
1.2.5 Montando tudo: O modelo matricial	13
1.2.6 Código para encontrar as matrizes	13
1.2.7 Código para encontrar os coeficientes	15
1.3 Interpretação dos coeficientes e multicolinearidade	18
1.3.1 Multicolinearidade	19
2 ESTIMAÇÃO E INFERÊNCIA	20
2.1 Teste de significância dos coeficientes: t-test e F-test	20
3 DIAGNÓSTICO DO MODELO	25
3.1 Análise de resíduos em modelos múltiplos	25
3.2 Detecção de <i>outliers</i> e pontos de alavancagem.....	25
3.3 Multicolinearidade e fator de inflação da variância (VIF)	26
3.4 Exemplo numérico.....	27
4 IMPLEMENTAÇÃO EM PYTHON	28
4.1 Comparação entre modelos	29
4.2 Análise gráfica e métricas de desempenho.....	30
REFERÊNCIAS.....	32
GLOSSÁRIO	33

AJUSTE DE MODELOS LINEARES

1 REGRESSÃO LINEAR MÚLTIPLA

A regressão linear múltipla é uma técnica estatística que permite estudar como uma variável de interesse, chamada variável resposta, se relaciona com várias outras variáveis ao mesmo tempo. Enquanto a regressão linear simples utiliza apenas um fator para explicar ou prever um resultado, a versão múltipla permite considerar diversos fatores simultaneamente, o que aproxima o modelo de situações reais encontradas em ciência de dados, computação e engenharia.

Esse tipo de regressão é especialmente importante porque muitos fenômenos não dependem de um único elemento. Por exemplo, o tempo de resposta de um servidor pode depender do número de usuários simultâneos, do volume de dados trafegados e de características da infraestrutura. Ao incluir todas essas variáveis no modelo, obtemos estimativas mais precisas e uma visão mais clara de quais fatores realmente influenciam o comportamento do sistema.

O consumo energético de sistemas industriais e equipamentos não depende de um único fator, mas do comportamento conjunto de várias variáveis operacionais. Em unidades de refrigeração, máquinas industriais ou sistemas de automação, elementos como temperatura ambiente, carga aplicada, número de ciclos, tempo de operação, desgaste de componentes e eficiência dos motores impactam diretamente a demanda de energia. Ao ajustar um modelo de regressão múltipla, é possível estimar o consumo previsto para diferentes condições de operação, permitindo planejamento antecipado, identificar quais variáveis exercem maior influência (direcionando ações de melhoria), reduzir desperdícios, ao identificar condições de operação ineficientes, apoiar decisões de manutenção preventiva, substituição de peças e ajustes de máquinas para minimizar custos.

Modelos desse tipo são amplamente utilizados em indústrias, *data centers* e ambientes de automação, em que decisões orientadas por dados podem gerar economia significativa.

Outra aplicação clássica da regressão múltipla está no campo da precificação. Tanto para imóveis quanto para equipamentos industriais, diversos atributos

influenciam o valor final. Entre eles, podemos citar: área útil, número de ambientes, idade, localização, qualidade da construção, estado de conservação e indicadores econômicos no caso de imóveis. No caso de equipamentos, temos potência, tempo de uso, eficiência técnica e estado mecânico.

Com a regressão múltipla, é possível estimar preços de mercado com base em características observáveis, realizar simulações (“E se o imóvel tivesse mais um cômodo?” ou “E se o equipamento fosse mais novo?”), entender quais atributos agregam mais valor ao imóvel ou ao equipamento e automatizar avaliações, criando modelos escaláveis para sistemas imobiliários ou plataformas de leilão e revenda.

A regressão linear múltipla não é, portanto, apenas uma ferramenta teórica; ela é amplamente utilizada em diversos setores para apoiar decisões estratégicas, otimizar processos e gerar previsões confiáveis. A seguir, vamos entender alguns casos práticos em que esse tipo de modelagem desempenha um papel essencial.

1.1 Expansão para múltiplas variáveis explicativas

Quando analisamos apenas uma variável explicativa, o modelo linear simples é suficiente. No entanto, na maioria das aplicações reais, diversos fatores atuam simultaneamente sobre o resultado. Para capturar essa complexidade, ampliamos o modelo para incorporar vários preditores.

Matematicamente, representamos a variável resposta Y como dependente de várias variáveis $X_1, X_2, \dots, X_k$. O modelo passa a ser escrito como:

$$Y = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \dots + \beta_k X_k + \varepsilon$$

Nesse modelo:

- β_0 é o intercepto, ou seja, o valor esperado de Y quando todas as variáveis explicativas valem zero;
- $\beta_1, \beta_2, \dots, \beta_k$ são os coeficientes associados a cada variável explicativa;
- ε representa o erro aleatório, isto é, a parte do comportamento de Y que não foi explicada pelas variáveis do modelo.

Cada coeficiente β_i indica quanto o valor de Y tende a mudar quando a variável X_i aumenta uma unidade, mantendo todas as outras variáveis constantes. Esse

princípio, conhecido como *ceteris paribus*, é fundamental para interpretar corretamente os resultados, pois garante que estamos avaliando o efeito isolado de cada fator (Dionísio, 2023).

Ao investigar o desempenho de um servidor, considere o cenário de uma API utilizada por diversos clientes simultaneamente. A equipe de desenvolvimento deseja compreender como diferentes fatores de carga influenciam o tempo médio de resposta do sistema. Para isso, três variáveis são monitoradas em cada período de observação:

- O número de usuários simultâneos acessando a API (X_1);
- O volume médio de dados transmitidos por requisição, medido em megabytes (X_2);
- O tempo médio de resposta do servidor, em milissegundos (Y).

Com o objetivo de analisar essas relações de forma quantitativa, foram coletadas diversas medições dessas variáveis ao longo do tempo, resultando no seguinte conjunto de observações:

i	X_1 (usuários)	X_2 (dados_MB)	X_3 (tempo_ms)
1	10	2	120
2	20	4	150
3	30	6	180
4	40	8	210
5	50	10	240
6	60	12	270
7	70	14	300
8	80	16	330

Tabela 1 – Dados do desempenho de um servidor
Fonte: Elaborada pela autora (2025)

Para entender como diferentes fatores influenciam o tempo de resposta do servidor, podemos construir um modelo que relacione essa métrica com as variáveis monitoradas. Nesse caso, queremos prever o tempo médio de resposta ($tempo_ms$) a partir de dois elementos: o número de usuários simultâneos ($usuários$) e a quantidade de dados enviada em cada requisição ($dados_MB$).

Escrevemos esse modelo da seguinte forma:

$$tempo_ms = \beta_0 + \beta_1(n_usuarios) + \beta_2(dados_MB) + \varepsilon$$

Essa expressão indica que o tempo de resposta depende de três partes:

- Um valor inicial, representado por β_0 ;

- O efeito do número de usuários, representado por β_1 ;
- O efeito do volume de dados, representado por β_2 .

Quando ajustamos esse modelo aos dados observados (o desenvolvimento para encontrar os coeficientes será explicado em detalhes adiante), encontramos valores aproximados para esses coeficientes:

- $\beta_0 \approx 90$
- $\beta_1 \approx 2,88$
- $\beta_2 \approx 0,58$

Isso significa que a expressão pode ser reescrita como $tempo_ms = 90 + 2,88(n_usuarios) + 0,58(dados_MB) + \varepsilon$.

Esse exemplo evidencia como podemos combinar múltiplas variáveis para explicar ou prever o comportamento de um sistema computacional. Em vez de analisar cada fator separadamente, o modelo considera todos ao mesmo tempo, oferecendo uma visão mais completa da dinâmica do servidor.

A motivação para usar múltiplos preditores decorre da necessidade de modelos mais expressivos, capazes de representar fenômenos complexos e multifatoriais, comuns em aplicações de computação, ciência de dados e aprendizado de máquina.

1.2 Representação matricial

Para organizar melhor as informações do modelo e permitir cálculos eficientes, especialmente quando trabalhamos com muitas observações, representamos a regressão linear múltipla usando notação matricial. Essa forma compacta e estruturada facilita tanto a análise teórica quanto a implementação em linguagens de programação.

Vamos retomar o exemplo do servidor. Para cada instante observado, registramos:

- O número de usuários simultâneos;
- O volume médio de dados em megabytes;
- O tempo de resposta em milissegundos.

A partir desses dados, construímos quatro elementos fundamentais: o vetor Y , a matriz X , o vetor de parâmetros β e o vetor de erros ϵ .

1.2.1 Vetor Y

A variável resposta do modelo é o tempo médio de resposta do servidor (em milissegundos). Temos oito observações desse tempo, uma para cada combinação de número de usuários e volume de dados.

Por convenção, na regressão linear, representamos a variável resposta como um vetor coluna, ou seja, um vetor vertical. No nosso exemplo:

$$Y = [120, 150, 180, 210, 240, 270, 300, 330]^t$$

Para indicar um vetor coluna, usamos o símbolo de transposta (t), ou seja, convertemos um vetor linha (horizontal) em um vetor coluna (vertical). Essa organização é necessária para que a multiplicação matricial do modelo funcione corretamente.

1.2.2 Matriz X

A matriz X reúne todas as informações que utilizaremos para explicar ou prever a variável resposta Y . Ela funciona como um grande “painel”, em que cada linha corresponde a uma observação coletada e cada coluna representa uma das variáveis que acreditamos influenciar o comportamento do sistema. Assim, ao organizarmos os dados dessa forma, conseguimos transformar o problema em uma estrutura que pode ser manipulada por métodos de álgebra linear e por algoritmos de aprendizado de máquina.

No caso do exemplo do servidor, a matriz X precisa conter três colunas. A primeira é composta apenas por valores iguais a 1. Embora possa parecer estranha à primeira vista, essa coluna é essencial: ela representa o termo independente β_0 , também chamado de intercepto, permitindo que o modelo calcule o valor inicial de Y quando todas as variáveis explicativas são iguais a zero.

As duas colunas seguintes representam, respectivamente, o número de usuários simultâneos X_1 (n_{usuarios}) e o volume de dados trafegado por requisição

em megabytes X_2 (`dados_MB`). São essas variáveis que estamos investigando para compreender como afetam o tempo de resposta do servidor.

A matriz X , portanto, pode ser organizada da seguinte maneira:

Linha i	1	X_1 (<code>n_usuarios</code>)	X_2 (<code>dados_MB</code>)
1	1	10	2
2	1	20	4
3	1	30	6
4	1	40	8
5	1	50	10
6	1	60	12
7	1	70	14
8	1	80	16

Tabela 2– Exemplo de matriz X utilizada no modelo

Fonte: Elaborada pela autora (2025)

Para que você interprete essa tabela corretamente, é importante destacar alguns pontos:

- A coluna “Linha i ” não faz parte da matriz X ; ela existe apenas para indicar o número da observação;
- A coluna formada por 1s é indispensável, pois é ela que permite ao modelo incluir o intercepto β_0 ;
- Cada linha da matriz X deve estar perfeitamente alinhada com a linha correspondente do vetor Y , garantindo que cada combinação de variáveis explicativas esteja associada ao tempo de resposta correto.

1.2.3 Vetor β

No modelo de regressão linear múltipla, reunimos todos os coeficientes em um único vetor chamado β (beta). Esses coeficientes representam os efeitos que cada variável explicativa exerce sobre a variável resposta. Para o nosso exemplo, em que estamos estudando o tempo de resposta do servidor, o vetor β possui três coeficientes:

$$\beta = [\beta_0, \beta_1, \beta_2]^t$$

- $\beta_0 \rightarrow$ é o intercepto, isto é, o valor inicial de Y quando todas as variáveis explicativas são iguais a zero. Ele funciona como o “ponto de partida” do modelo;

- $\beta_1 \rightarrow$ indica quanto o tempo de resposta tende a aumentar (ou diminuir) quando adicionamos um usuário simultâneo, mantendo todas as demais variáveis fixas;
- $\beta_2 \rightarrow$ mostra como o tempo de resposta muda quando o volume de dados enviados cresce em 1MB, também mantendo os outros fatores constantes.

Esses valores ainda não são conhecidos; o objetivo do processo de modelagem é justamente estimá-los a partir dos dados observados. A seguir, veremos como o Python, por meio da biblioteca *scikit-learn*, realiza essa estimação de maneira eficiente.

1.2.4 Vetor ε

O vetor ε contém a parte da variável resposta que não conseguimos explicar por meio das variáveis X_1 e X_2 . Mesmo quando escolhemos bons preditores, sempre existirão pequenas diferenças entre o valor previsto pelo modelo e o valor realmente observado. Essas diferenças são chamadas de erros, resíduos ou perturbações aleatórias, e são representadas pelo vetor ε .

Escrevemos esse vetor como:

$$\varepsilon = [\varepsilon_1, \varepsilon_2, \dots, \varepsilon_8]^t$$

Cada ε_i representa o “desvio” da i -ésima observação.

$$\varepsilon_i = \text{valor real observado} - \text{valor previsto pelo modelo}$$

Esses desvios existem por vários motivos:

- Porque nem todas as variáveis que influenciam Y foram incluídas no modelo;
- Porque há flutuações naturais nos dados (ruído, variabilidade ou imprecisões);
- Porque todo modelo é uma aproximação da realidade.

Em outras palavras, o vetor ε representa a parte de Y que o modelo não consegue capturar apenas com X_1 e X_2 . Entender e analisar esses erros será essencial nas etapas de diagnóstico, quando verificaremos se o modelo está funcionando adequadamente.

1.2.5 Montando tudo: O modelo matricial

Reunindo todas essas peças, o modelo completo pode ser escrito como:

$$Y = X\beta + \varepsilon$$

Essa expressão é extremamente poderosa porque:

- Organiza todo o modelo em uma única equação;
- Permite usar álgebra linear para encontrar a solução ótima;
- É baseada na mesma estrutura usada por bibliotecas profissionais de Python (como *NumPy*, *scikit-learn* e *statsmodels*);
- Facilita muito a compreensão do método dos mínimos quadrados.

Em outras palavras, essa forma matricial é a ponte entre a teoria de regressão linear e a prática computacional usada em aplicações reais de ciência de dados e aprendizado de máquina.

Para tornar essa representação mais concreta e mostrar como ela aparece na prática computacional, podemos construir a matriz X e o vetor Y diretamente em Python, utilizando bibliotecas amplamente usadas em ciência de dados. O código “Montando X e Y ” transforma exatamente os dados apresentados na tabela “Exemplo de matriz X utilizada no modelo” em seus formatos matriciais: Y como vetor coluna e X como matriz contendo a coluna de intercepto e as variáveis explicativas. Essa implementação permite visualizar, de maneira explícita, como a teoria da formulação matricial se traduz em estruturas manipuláveis por algoritmos de aprendizagem de máquina.

1.2.6 Código para encontrar as matrizes

Agora que organizamos nossos dados na forma matricial, podemos utilizar essa estrutura para estimar os coeficientes do modelo. A representação por matrizes não é apenas uma formalidade matemática: ela é exatamente o que permite aplicar o método dos mínimos quadrados e também o que sustenta as funções modernas de regressão disponíveis em Python.

Para mostrar como essa teoria se transforma em prática, o código a seguir constrói a matriz X e o vetor Y usando *NumPy* e *pandas*. Essa etapa é fundamental para que possamos, nos passos seguintes, calcular os coeficientes do modelo e observar como o Python implementa, de forma automatizada, os mesmos princípios vistos em álgebra linear.

```
import numpy as np
import pandas as pd

# Dados do exemplo
dados = pd.DataFrame({
    "n_usuarios": [10, 20, 30, 40, 50, 60, 70, 80],
    "dados_MB":   [2,  4,  6,  8, 10, 12, 14, 16],
    "tempo_ms":   [120, 150, 180, 210, 240, 270, 300, 330]
})

# Vetor Y (n x 1)
Y = dados["tempo_ms"].values.reshape(-1, 1)

# Matriz X (n x (k+1)) com coluna de 1's + preditores
n = len(dados)
coluna_ones = np.ones((n, 1))
X = np.hstack([
    coluna_ones,
    dados[["n_usuarios", "dados_MB"]].values
])

print("Matriz X:")
print(X)
print("\nVetor Y:")
print(Y)
```

Código-fonte 1 – Montando X e Y
Fonte: Elaborado pelo autor (2025)

Esse código gera o seguinte resultado encontrado na figura “Resultado do código-fonte 1”.

```
Matriz X:  
[[ 1. 10.  2.]  
 [ 1. 20.  4.]  
 [ 1. 30.  6.]  
 [ 1. 40.  8.]  
 [ 1. 50. 10.]  
 [ 1. 60. 12.]  
 [ 1. 70. 14.]  
 [ 1. 80. 16.]]  
  
Vetor Y:  
[[120]  
 [150]  
 [180]  
 [210]  
 [240]  
 [270]  
 [300]  
 [330]]
```

Figura 1 – Resultado do código-fonte 1
Fonte: Elaborada pela autora (2025)

A execução desse código confirma, de forma visual, a estrutura que discutimos conceitualmente: Y aparece como um vetor coluna contendo os tempos observados, enquanto X reúne a coluna de 1s responsável pelo intercepto e as colunas com os valores de `n_usuarios` e `dados_MB`. A saída do programa, apresentada na figura, ilustra claramente como os dados originais são reorganizados em formato matricial, preparando o terreno para a aplicação do método dos mínimos quadrados e para o uso de algoritmos de regressão disponíveis nas bibliotecas de Python.

Essa representação é fundamental, pois conecta estatística, programação e técnicas de otimização, formando a base teórica de muitos algoritmos de aprendizado supervisionado.

1.2.7 Código para encontrar os coeficientes

Uma vez construída a matriz X e o vetor Y, estamos prontos para estimar os coeficientes do modelo de regressão. Do ponto de vista teórico, vimos que o método dos mínimos quadrados fornece a seguinte solução matricial:

$$\hat{\beta} = (X^T X)^{-1} X^T Y$$

Essa fórmula permite compreender como a regressão linear múltipla é construída matematicamente. No entanto, na prática computacional, calcular diretamente a inversa de $(X^T X)$ pode gerar instabilidade numérica, especialmente quando as variáveis explicativas são correlacionadas. Por esse motivo, bibliotecas modernas utilizam métodos mais robustos, como a função `lstsq` do *NumPy*, que calcula uma solução equivalente aos mínimos quadrados sem a necessidade de inverter matrizes. Nos exemplos a seguir, utilizaremos essa abordagem mais estável.

Em seguida, mostraremos como obter o mesmo resultado de forma automatizada com a biblioteca *scikit-learn*.

```
import numpy as np
import pandas as pd

# Dados do exemplo
dados = pd.DataFrame({
    "n_usuarios": [10, 20, 30, 40, 50, 60, 70, 80],
    "dados_MB": [2, 4, 6, 8, 10, 12, 14, 16],
    "tempo_ms": [120, 150, 180, 210, 240, 270, 300, 330]
})

# Vetor Y (n x 1)
Y = dados["tempo_ms"].values.reshape(-1, 1)

# Matriz X (n x (k+1)) com coluna de 1's + preditores
n = len(dados)
coluna_ones = np.ones((n, 1))
X = np.hstack([
    coluna_ones,
    dados[["n_usuarios", "dados_MB"]].values
])

# Cálculo dos coeficientes por mínimos quadrados usando
# lstsq (mais estável)
beta_hat, residuals, rank, s = np.linalg.lstsq(X, Y,
rcond=None)

print("Coeficientes estimados ( $\beta^{\wedge}$ ):")
print(beta_hat)
```

Código-fonte 2 – Encontrando os coeficientes
Fonte: Elaborado pelo autor (2025)

O código apresentado utiliza o método dos mínimos quadrados para estimar os coeficientes do modelo de regressão linear múltipla. Na figura a seguir podemos visualizar o resultado desse código.


```
Coeficientes estimados ( $\hat{\beta}$ ):  
[[90.  
 [ 2.88461538  
 [ 0.57692308]]
```

Figura 2 – Resultado do código-fonte 2
Fonte: Elaborada pela autora (2025)

Em vez de calcular diretamente a inversa de $X^T X$, o que pode gerar instabilidade numérica em muitos conjuntos de dados, empregamos a função `np.linalg.lstsq`, que resolve o problema de mínimos quadrados de forma robusta e eficiente. Esse método encontra o vetor $\hat{\beta}$ que minimiza a soma dos erros quadráticos, produzindo uma solução equivalente à fórmula teórica dos mínimos quadrados, mas sem a necessidade de inverter matrizes. A função `lstsq` retorna o vetor de coeficientes do modelo na ordem $[\beta_0, \beta_1, \beta_2]^t$, correspondendo, respectivamente, ao intercepto, ao efeito do número de usuários simultâneos e ao efeito do volume de dados sobre o tempo de resposta do servidor. Dessa forma, o código traduz diretamente a teoria apresentada anteriormente para uma implementação computacional estável e alinhada às melhores práticas computacionais.

Em um contexto de ciência de dados, não é comum implementar manualmente a solução matricial da regressão linear, pois bibliotecas especializadas já fornecem algoritmos otimizados e altamente estáveis. A biblioteca *scikit-learn*, amplamente utilizada em *machine learning*, disponibiliza a classe `LinearRegression`, que calcula automaticamente os coeficientes do modelo utilizando uma versão numericamente robusta do método dos mínimos quadrados.

O código a seguir mostra como ajustar o modelo diretamente com *scikit-learn*.

```
from sklearn.linear_model import LinearRegression  
  
# X_pred contém apenas os preditores (sem a coluna de 1's)  
X_pred = dados[["n_usuarios", "dados_MB"]].values  
Y_resp = dados["tempo_ms"].values  
  
modelo = LinearRegression() # Por padrão,  
fit_intercept=True  
modelo.fit(X_pred, Y_resp)  
  
print("Intercepto ( $\beta_0$ ):", modelo.intercept_)  
print("Coeficientes ( $\beta_1, \beta_2$ ):", modelo.coef_)
```

Código-fonte 3 – Encontrando os coeficientes com *scikit-learn*
Fonte: Elaborado pelo autor (2025)

Na implementação com *scikit-learn*, não precisamos criar manualmente a coluna de 1s, pois o próprio modelo faz isso internamente quando `fit_intercept=True`. Na figura abaixo podemos observar a impressão desse código.

```
Intercepto ( $\beta_0$ ): 90.00000000000003  
Coeficientes ( $\beta_1, \beta_2$ ): [2.88461538 0.57692308]
```

Figura 3 – Resultado do código-fonte 3
Fonte: Elaborada pela autora (2025)

Depois de ajustado, o modelo armazena:

- O intercepto no atributo ***intercept_***
- Os coeficientes das variáveis explicativas no atributo ***coef_***.

Ao comparar esses valores com os coeficientes obtidos pelo método matricial apresentado anteriormente, pode-se observar que os resultados são equivalentes. A diferença está apenas na forma de cálculo: no primeiro caso, utilizamos explicitamente a fórmula teórica da álgebra linear; no segundo, empregamos uma implementação numérica otimizada e amplamente utilizada em aplicações reais de *machine learning*.

1.3 Interpretação dos coeficientes e multicolinearidade

Quando ajustamos um modelo de regressão linear múltipla, cada coeficiente β_i representa o efeito isolado de uma variável explicativa sobre a variável resposta Y. Em outras palavras, β_i indica quanto esperamos que Y varie quando aumentamos X_i em uma unidade, mantendo todas as outras variáveis constantes.

No exemplo numérico, obtivemos aproximadamente:

$$tempo_{ms} = 90 + 2,88(n_{usu\acute{a}rios}) + 0,58(dados_{MB})$$

Vamos interpretar cada coeficiente de forma clara:

- $\beta_0 \approx 90$ representa o tempo base de resposta do servidor. Mesmo com zero usuários e sem dados trafegados, o sistema ainda apresenta um tempo

mínimo em torno de 90 ms. Esse valor corresponde ao custo “fixo” de funcionamento do servidor;

- $\beta_1 \approx 2,88$ indica que, se o número de usuários simultâneos aumentar em uma unidade, o tempo médio de resposta tende a crescer cerca de 2,88 ms, desde que o volume de dados por requisição permaneça o mesmo. Esse coeficiente captura o impacto da carga de usuários no desempenho;
- $\beta_2 \approx 0,58$ mostra que, para cada 1MB adicional transmitido por requisição, o tempo de resposta aumenta aproximadamente 0,58 ms, assumindo que o número de usuários não se altere. Ele indica quanto o tráfego de dados afeta o desempenho.

Essas interpretações nos ajudam a entender como cada fator contribui individualmente para o comportamento do sistema, permitindo diagnósticos mais precisos e decisões mais informadas.

1.3.1 Multicolinearidade

A interpretação dos coeficientes só é válida quando as variáveis explicativas trazem informações diferentes para o modelo. Quando duas ou mais variáveis estão altamente correlacionadas, por exemplo: se sempre que o número de usuários aumenta o volume de dados também aumenta na mesma proporção, o modelo tem dificuldade em separar o efeito de cada uma delas.

Esse fenômeno é chamado de multicolinearidade e pode causar:

- Coeficientes instáveis (pequenas mudanças nos dados produzem grandes mudanças nos valores de β);
- Sinais inesperados (coeficiente positivo quando esperávamos negativo ou vice-versa);
- Dificuldade em interpretar efeitos individuais;
- Aumento da variância dos estimadores, reduzindo sua confiabilidade.

Nessas situações, o modelo até consegue fazer previsões boas, mas perde precisão ao tentar explicar qual variável causa qual efeito, porque elas estão “contando a mesma história”.

Compreender o significado de cada coeficiente é um passo essencial para interpretar os resultados da regressão múltipla. No entanto, antes de confiar nas conclusões ou utilizá-las em um processo decisório, precisamos avaliar se esses coeficientes foram estimados de maneira precisa e se realmente representam um efeito significativo no fenômeno estudado. É nesse ponto que entram as etapas de estimação e inferência estatística, nas quais verificamos como os parâmetros são obtidos a partir dos dados e analisamos a confiabilidade dessas estimativas por meio de testes e medidas de incerteza.

2 ESTIMAÇÃO E INFERÊNCIA

Construir o modelo é apenas o ponto de partida. Depois de definir a estrutura da regressão e compreender o papel de cada variável, precisamos estimar os valores dos coeficientes β , que determinam como a variável resposta se comporta diante das variáveis explicativas. Esse processo, chamado de estimação, busca encontrar os valores de β que melhor se ajustam aos dados observados, minimizando o erro entre o que o modelo prevê e o que realmente ocorreu.

A etapa seguinte é a inferência estatística, que responde a perguntas fundamentais para qualquer análise: os coeficientes encontrados são realmente relevantes ou poderiam ter surgido apenas por acaso, devido à variabilidade natural dos dados? Por meio de testes estatísticos e medidas de incerteza, a inferência permite avaliar a confiabilidade das estimativas, ajudando a distinguir efeitos reais de ruído aleatório. Dessa forma, estimação e inferência trabalham juntas para garantir que o modelo não apenas se ajuste bem, mas também produza resultados sólidos e interpretáveis.

2.1 Teste de significância dos coeficientes: t-test e F-test

Após estimarmos os coeficientes do modelo, o próximo passo é verificar se eles realmente exercem influência sobre a variável resposta ou se os valores obtidos podem ter surgido devido apenas ao acaso. Essa análise é feita por meio de testes de significância.

O teste t examina cada coeficiente individualmente. Ele responde à pergunta: “A variável X_i contribui de forma estatisticamente relevante para explicar Y ?”. Se o teste indicar que o coeficiente não é significativamente diferente de zero, isso sugere que aquela variável pode não estar desempenhando um papel importante no modelo.

Já o teste F avalia o modelo como um todo. Ele compara o desempenho do modelo completo, aquele que utiliza todas as variáveis explicativas, com o desempenho de um modelo mais simples que contém apenas o intercepto. Em outras palavras, o teste F responde: “O conjunto das variáveis explicativas melhora a capacidade do modelo de explicar Y ?”.

Esses testes complementam a etapa de estimação ao fornecer uma medida de confiança sobre os coeficientes obtidos. Eles ajudam a identificar quais variáveis realmente adicionam informação ao modelo e se o ajuste global é estatisticamente consistente com os dados observados.

Após ajustar o modelo de regressão, podemos utilizar Python para realizar automaticamente os testes t e F, que verificam a significância dos coeficientes e do modelo como um todo. Para isso, utilizamos a biblioteca *statsmodels*, que fornece uma implementação completa dos procedimentos estatísticos associados à regressão linear.

O código abaixo ajusta o modelo e, em seguida, exibe um resumo estatístico contendo os testes t e F:

```
import statsmodels.api as sm

# Construção da matriz X (com intercepto)
X_pred = dados[["n_usuarios", "dados_MB"]]
X_pred = sm.add_constant(X_pred)    # adiciona a coluna de
1s automaticamente

# Vetor Y
Y_resp = dados["tempo_ms"]

# Ajuste do modelo OLS (Ordinary Least Squares)
modelo = sm.OLS(Y_resp, X_pred).fit()

# Exibindo o resumo do modelo com testes t e F
print(modelo.summary())
```

Código-fonte 4 – Teste t e F
Fonte: Elaborado pelo autor (2025)

Ao executar esse código, o método `summary()` gera uma tabela completa contendo:

- **Teste t** para cada coeficiente, incluindo:
 - Estimativa do coeficiente β ;
 - Erro padrão;
 - Valor t;
 - P-valor;
 - Intervalo de confiança.

Esses resultados permitem avaliar se cada variável explicativa contribui significativamente para explicar a variável resposta.

- **Teste F** para o modelo como um todo, incluindo:
 - Estatística F (*F-statistic*);
 - P-valor associado (*Prob (F-statistic)*);

Esse teste indica se, coletivamente, as variáveis explicativas melhoram o ajuste do modelo em comparação com um modelo que contém apenas o intercepto.

Na figura abaixo podemos observar o resultado do código para o exemplo inicial desse capítulo.

```

=====
                        OLS Regression Results
=====
Dep. Variable:          tempo_ms      R-squared:                1.000
Model:                  OLS          Adj. R-squared:           1.000
Method:                 Least Squares  F-statistic:             5.297e+30
Date:                  Mon, 17 Nov 2025  Prob (F-statistic):      4.54e-91
Time:                  19:55:01        Log-Likelihood:          230.62
No. Observations:      8              AIC:                    -457.2
Df Residuals:          6              BIC:                    -457.1
Df Model:              1
Covariance Type:       nonrobust
=====

```

	coef	std err	t	P> t	[0.025	0.975]
const	90.0000	6.58e-14	1.37e+15	0.000	90.000	90.000
n_usuarios	2.8846	1.25e-15	2.3e+15	0.000	2.885	2.885
dados_MB	0.5769	2.51e-16	2.3e+15	0.000	0.577	0.577

```

=====
Omnibus:                0.538      Durbin-Watson:           0.340
Prob(Omnibus):          0.764      Jarque-Bera (JB):         0.499
Skew:                   0.218      Prob(JB):                 0.779
Kurtosis:               1.857      Cond. No.                 3.37e+16
=====

```

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

[2] The smallest eigenvalue is 1.87e-29. This might indicate that there are strong multicollinearity problems or that the design matrix is singular.

Figura 4 – Resultado do código-fonte 4

Fonte: Elaborada pela autora (2025)

A saída apresentada pelo comando `modelo.summary()` resume todas as informações estatísticas relevantes do ajuste do modelo de regressão linear múltipla. O relatório indica, antes de tudo, que a variável dependente modelada é `tempo_ms` e que o método utilizado foi o OLS (*Ordinary Least Squares*), o procedimento clássico para estimar os coeficientes da regressão. Logo no início, observamos que o valor de *R-squared* é igual a 1.000, o que significa que o modelo explica 100% da variação nos tempos de resposta observados. Esse resultado é esperado, pois os dados utilizados seguem uma relação perfeitamente linear, sem qualquer ruído ou variabilidade.

O relatório também apresenta o *Adjusted R-squared*, cujo valor é 1.000, confirmando que, mesmo após ajustar pelo número de variáveis no modelo, o poder explicativo continua perfeito. Em seguida, o valor extremamente elevado da *F-statistic*, acompanhado de um p-valor praticamente igual a zero (*Prob (F-statistic)* $\approx 4.54e-91$), indica que o conjunto das variáveis explicativas, número de usuários e volume de dados, melhora significativamente o ajuste do modelo em comparação com um modelo que teria apenas o intercepto. Em outras palavras, o modelo completo é estatisticamente superior e altamente significativo.

A tabela de coeficientes detalha a contribuição de cada variável. O intercepto (`const`) tem valor estimado igual a 90, representando o tempo base de resposta quando não há usuários e nenhum dado sendo trafegado. Os coeficientes de `n_usuarios` (cerca de 2.8846) e `dados_MB` (aproximadamente 0.5769) possuem valores de `t` extremamente altos e `p`-valores de 0.000, evidenciando que essas variáveis influenciam fortemente a variável resposta. Além disso, os intervalos de confiança são extremamente estreitos, reforçando que a estimativa dos coeficientes é praticamente exata, fato explicado pela ausência de ruído nos dados.

Na parte final da saída, são apresentados alguns testes adicionais de diagnóstico. Os valores de *Omnibus*, *Prob(Omnibus)*, *Jarque-Bera* e seus respectivos `p`-valores indicam que não há evidências de violação séria da normalidade dos resíduos, embora o pequeno número de observações limite a interpretação desses testes. O índice *Durbin-Watson* assume valor 0.340, sugerindo autocorrelação positiva dos resíduos; porém, com dados perfeitamente lineares e sem ruído, essa medida perde relevância prática. Por fim, o *Condition Number*, extremamente alto ($3.37e+16$), sinaliza que há forte multicolinearidade entre as variáveis explicativas. De fato, no conjunto de dados utilizado, o número de usuários e volume de dados aumentam sempre juntos, o que faz com que o modelo encontre dificuldade em separar o efeito individual de cada variável.

Em síntese, o relatório apresenta um modelo com ajuste perfeito devido à construção artificial dos dados, coeficientes altamente significativos e indicadores diagnósticos que refletem tanto a ausência de ruído quanto a forte colinearidade entre as variáveis. Essa saída ilustra bem como interpretar os principais indicadores de uma regressão múltipla e reforça a importância de analisar tanto o ajuste quanto a estrutura dos dados antes de tirar conclusões.

Assim, com apenas algumas linhas de código, o Python realiza automaticamente todos os cálculos necessários para avaliar a significância estatística do modelo e dos coeficientes estimados. Essa abordagem é amplamente utilizada em ciência de dados por sua praticidade, precisão e aderência às melhores práticas de modelagem estatística.

3 DIAGNÓSTICO DO MODELO

Depois de ajustar o modelo de regressão, o próximo passo é verificar se ele realmente representa bem os dados. Um modelo pode até gerar bons coeficientes e parecer adequado à primeira vista, mas isso não garante que todas as suas suposições estejam satisfeitas. Por isso, a etapa de diagnóstico do modelo é essencial: ela permite identificar inconsistências, padrões inesperados e possíveis problemas que comprometem a qualidade das previsões.

3.1 Análise de resíduos em modelos múltiplos

Os resíduos representam a parte do comportamento de Y que o modelo não conseguiu explicar. Eles são calculados como resíduo $i = y_i - \hat{y}_i$ e funcionam como um “termômetro” da qualidade do ajuste. Para que o modelo seja considerado adequado, os resíduos devem apresentar um comportamento aleatório, sem padrões visíveis. Em geral, esperamos que:

- Tenham média próxima de zero, indicando ausência de viés sistemático;
- Apresentem variância aproximadamente constante (homocedasticidade);
- Não exibam padrões, tendências ou correlação ao longo das observações;
- Sigam um comportamento aproximadamente normal, principalmente para fins de inferência.

Gráficos como resíduos *versus* valores previstos, histograma dos resíduos e gráfico de resíduos padronizados são ferramentas fundamentais para verificar essas características. Caso algum padrão seja identificado, isso indica que o modelo pode estar violando premissas importantes.

3.2 Detecção de *outliers* e pontos de alavancagem

Em qualquer análise real, alguns pontos podem se comportar de forma diferente do restante do conjunto de dados. Eles merecem atenção especial:

- *Outliers* são observações que fogem completamente do padrão da maioria dos dados. Eles podem distorcer o ajuste do modelo e influenciar a interpretação dos coeficientes;
- Pontos de alavancagem são valores extremos nas variáveis explicativas. Mesmo que não sejam *outliers* no valor de Y , podem exercer forte influência na inclinação do modelo, “puxando” a reta de regressão em sua direção.

Para identificar esses pontos, utilizamos medidas clássicas como:

- Alavancagem (h_{ii}), que indica quanto cada ponto contribui para o ajuste;
- Distância de Cook, que mede o impacto da remoção de cada observação no modelo;
- Resíduos studentizados, que ajudam a detectar valores anômalos em Y .

Reconhecer esses pontos é fundamental, pois eles podem distorcer conclusões e prejudicar o desempenho preditivo.

3.3 Multicolinearidade e fator de inflação da variância (VIF)

A multicolinearidade, como explicado anteriormente, ocorre quando duas ou mais variáveis explicativas estão fortemente correlacionadas entre si. Nessa situação, o modelo encontra dificuldade para isolar o efeito individual de cada variável, resultando em coeficientes instáveis e imprecisos.

Uma forma amplamente utilizada para medir essa relação é o Fator de Inflação da Variância (VIF), definido por:

$$VIF_j = \frac{1}{1 - R_j^2}$$

Aqui, R_j^2 é o coeficiente de determinação obtido ao regressar X_j contra todas as outras variáveis explicativas. Valores elevados de VIF indicam que o preditor fornece informação redundante e pode estar prejudicando a interpretação do modelo. Em geral:

- $VIF < 5 \rightarrow$ multicolinearidade baixa (situação aceitável);
- VIF entre 5 e 10 $\rightarrow$ atenção necessária;

- $VIF > 10 \rightarrow$ forte multicolinearidade, recomenda-se reavaliar as variáveis.

A análise conjunta dos resíduos, *outliers* e multicolinearidade fornece uma visão completa da qualidade do modelo e ajuda a identificar caminhos para aprimorar sua performance e confiabilidade.

3.4 Exemplo numérico

Para consolidar os conceitos de diagnóstico do modelo apresentados anteriormente, vamos analisar novamente o conjunto de dados utilizado no estudo de desempenho do servidor. Relembrando: a equipe de desenvolvimento acompanha três variáveis essenciais para entender o comportamento da API - o número de usuários simultâneos (X_1), o volume médio de dados transmitidos por requisição, em megabytes (X_2) e o tempo médio de resposta do servidor, em milissegundos (Y).

Após ajustar o modelo, obtivemos a equação estimada:

$$\hat{Y} = 90 + 2,88X_1 + 0,58X_2$$

Para encontrar os resíduos, devemos calcular a diferença entre o valor observado e o valor previsto:

$$\varepsilon_i = Y_i - \hat{Y}_i.$$

Nesse conjunto de dados, todos os resíduos são iguais a zero. Isso ocorre porque as variáveis seguem uma relação perfeitamente linear, sem ruído ou variação aleatória. Embora esse cenário seja irrealista, ele é didaticamente útil para mostrar como um modelo se comporta quando ajusta perfeitamente os dados.

Em relação a *outliers* e pontos de alavancagem, ao analisar a estrutura do exemplo, não há valores distantes do padrão geral nem observações extremas em X_1 ou X_2 . Portanto, compreendemos que não existem *outliers* em Y e não existem pontos de alavancagem fortes em X_1 ou X_2 . Esse resultado reforça o fato de que a regressão está sendo aplicada sobre um conjunto de dados idealmente estruturado.

Para fins de comparação, se introduzíssemos artificialmente um ponto como (usuários = 200, dados = 40, tempo = 600), ele seria simultaneamente um *outlier* e um

ponto de alavancagem, afetando significativamente o ajuste. Tal exemplo pode ser explorado posteriormente para demonstrar o impacto dessas observações no modelo.

Embora não existam resíduos ou *outliers*, o conjunto apresenta um problema sério no nível das variáveis explicativas: multicolinearidade extrema. Observe que $X_2 = 0,2X_1$. Essa relação perfeita torna impossível distinguir o efeito exclusivo de cada preditor, uma vez que ambos carregam essencialmente a mesma informação. Isso se reflete nos valores do VIF (*Variance Inflation Factor*), que aparecem extremamente elevados devido à colinearidade total entre X_1 e X_2 .

Em aplicações reais, multicolinearidade tão intensa pode tornar os coeficientes instáveis, dificultar a interpretação dos efeitos individuais, aumentar erros padrão e comprometer a análise inferencial. Mesmo que o modelo apresente ajuste perfeito, o diagnóstico revela que é interpretativamente frágil, pois os preditores são redundantes.

Esse exemplo numérico evidencia três comportamentos importantes: resíduos exatamente nulos mostram como o modelo se comporta diante de dados perfeitamente lineares, a ausência de *outliers* e pontos de alavancagem reforça que o conjunto é homogêneo e bem comportado e a multicolinearidade severa demonstra que um modelo pode prever bem, mas ainda assim apresentar problemas estruturais que comprometem a interpretação dos coeficientes.

Esse diagnóstico ilustra, de forma integrada, como analisar e interpretar a qualidade de um modelo de regressão, conectando teoria estatística, comportamento dos dados e raciocínio computacional.

4 IMPLEMENTAÇÃO EM PYTHON

A etapa de implementação é essencial para transformar a teoria da regressão múltipla em aplicações reais dentro de sistemas de aprendizado de máquina. Em ambientes computacionais, o ajuste de modelos costuma ser integrado a *pipelines* de análise, monitoramento e tomada de decisão. Python é atualmente uma das ferramentas mais utilizadas nesse contexto, graças à clareza da linguagem e à disponibilidade de bibliotecas robustas.

A biblioteca *scikit-learn* é amplamente utilizada em ciência de dados por oferecer implementações eficientes e estáveis dos principais algoritmos supervisionados. Para ajustar um modelo de regressão múltipla, utilizamos a classe `LinearRegression`, que aplica internamente o método dos mínimos quadrados. Além disso, essa classe estima automaticamente o intercepto e os coeficientes do modelo. Uma vez ajustado, o objeto modelo permite acessar previsões, avaliar desempenho e extrair informações relevantes para interpretação estatística.

4.1 Comparação entre modelos

Após ajustar o modelo de regressão múltipla, é fundamental avaliar sua qualidade preditiva. As métricas numéricas mais utilizadas em problemas de regressão são o Erro Quadrático Médio (MSE), o Erro Absoluto Médio (MAE) e o Coeficiente de Determinação (R^2). Essas medidas complementam a análise gráfica e fornecem evidências objetivas sobre o desempenho do modelo:

- MSE (*Mean Squared Error*) mede a média dos erros ao quadrado, dando mais peso a desvios maiores;
- MAE (*Mean Absolute Error*) calcula a média das diferenças absolutas entre valores reais e previstos;
- R^2 indica a proporção da variabilidade de Y explicada pelo modelo; quanto mais próximo de 1, melhor o ajuste.

O código a seguir mostra como calcular essas métricas utilizando a biblioteca *scikit-learn*.

```
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error,
mean_absolute_error, r2_score

# Separando preditores e resposta
X_pred = dados[["n_usuarios", "dados_MB"]].values
Y_resp = dados["tempo_ms"].values

# Ajuste do modelo
modelo = LinearRegression()
modelo.fit(X_pred, Y_resp)

# Previsões
Y_pred = modelo.predict(X_pred)
```

```
# Cálculo das métricas
mse = mean_squared_error(Y_resp, Y_pred)
mae = mean_absolute_error(Y_resp, Y_pred)
r2 = r2_score(Y_resp, Y_pred)

print("MSE:", mse)
print("MAE:", mae)
print("R²:", r2)
```

Código-fonte 5 – Código com as métricas
Fonte: Elaborado pelo autor (2025)

Na figura a seguir podemos ver a impressão do código anterior.

```
MSE: 2.0194839173657902e-28
MAE: 7.105427357601002e-15
R²: 1.0
```

Figura 5 – Resultado do código-fonte 5
Fonte: Elaborada pela autora (2025)

As métricas de desempenho mostram que o ajuste do modelo foi praticamente perfeito: o R^2 igual a 1 indica que toda a variabilidade do tempo de resposta é explicada pelos preditores, e os valores extremamente pequenos de MSE e MAE refletem erros numéricos insignificantes. Isso ocorre porque o conjunto de dados utilizado é perfeitamente linear, sem ruído nem variação aleatória, o que resulta em previsões exatas por parte do modelo.

As métricas de desempenho indicam quanto o modelo se ajusta aos dados observados. O MSE mede o erro quadrático médio, penalizando grandes discrepâncias, o MAE avalia o erro absoluto médio, oferecendo uma medida mais robusta a *outliers* e o R^2 indica a proporção da variância de Y explicada pelo modelo. Em conjunto, essas métricas permitem identificar modelos mais eficientes, comparar diferentes especificações e avaliar se o ajuste está alinhado aos objetivos analíticos da aplicação.

4.2 Análise gráfica e métricas de desempenho

Além das métricas numéricas, é fundamental realizar uma análise visual para verificar se o modelo está se comportando adequadamente. Gráficos simples podem revelar padrões que passam despercebidos em valores estatísticos.

Entre os gráficos mais utilizados estão:

- Dispersão entre valores reais e previstos, que mostra a qualidade geral do ajuste;
- Gráficos de resíduos, úteis para detectar padrões, heterocedasticidade e erros estruturais;
- Curvas de erro ou *learning curves*, que indicam problemas de *overfitting* ou *underfitting*;
- Histogramas ou distribuições dos resíduos, que auxiliam na verificação de normalidade.

Essas representações gráficas complementam as métricas tradicionais, oferecendo uma visão qualitativa do comportamento do modelo e contribuindo para decisões mais informadas e robustas.

REFERÊNCIAS

BUSSAB, Wilton de Oliveira; MORETTIN, Pedro Alberto. **Estatística Básica**. 8. ed. São Paulo: Saraiva, 2015.

DUBINSKI, Juliana; JOMAR, Paulo. **Regressão Linear Simples – Exercício Experimental**. [S.l.]: [s.n.], s.d.

GONZALEZ, Manoel. **Estatística Aplicada à Engenharia e à Computação**. 3. ed. São Paulo: Pearson, 2018.

GUIMARÃES, Paulo Roberto Barbosa. **Análise de Regressão**. Curitiba: Universidade Federal do Paraná, 2011.

IZBICKI, Rafael; MENDONÇA, Tiago. **Machine Learning sob a ótica estatística: uma abordagem preditivista para a estatística com exemplos em R**. São Carlos: UFSCar/Insper, 2017.

MAIA, Alexandre Gori. **Econometria: conceitos e aplicações**. Campinas: Editora Unicamp, 2017.

MARQUES, Paulo C.; LOPES, Hedibert F. **Fundamentos de Aprendizagem Estatística: Regressão**. São Paulo: FGV, 2017.

MONTGOMERY, Douglas C.; RUNGER, George C. **Estatística Aplicada e Probabilidade para Engenheiros**. 6. ed. Rio de Janeiro: LTC, 2016.

VALLE, Flávio da Cunha; REIS, Elizabeth dos; MIGON, Helio S. **Introdução à Inferência Bayesiana**. 2. ed. Rio de Janeiro: SBM, 2020.

GLOSSÁRIO

API	<i>Application Programming Interface</i> . É um conjunto de regras, padrões e ferramentas que permite que diferentes sistemas, aplicações ou serviços comuniquem-se entre si.
------------	---